AutoMitra — Vehicle Service Center Management System

A Centralized Relational Web Application with Authoritative Billing, Atomic Inventory Deductions, Restrictive Historical Data Preservation, and Automated Test Verification

Project Title	AutoMitra — Vehicle Service Center Management System
Author / Contributor	Krrish Prajapati (@krrishprajapati1111) — Mechatronics Engineering
Target Currency	Indian Rupees (■ INR) Localization
Software Architecture	3-Tier MVC Web Application (Flask 3.1, SQLAlchemy 2.0, Flask-WTF)
Database Engines	MySQL 8.0+ / MariaDB 10.4+ (InnoDB Engine) & Embedded SQLite
Verified Test Suite	53 Tests Passed (0 Failed, 0 Skipped, 0 Warnings, 100% CI Green)
GitHub Repository	https://github.com/AdigarlaRakesh-1/Automitra (v1.0.0 Release)

1. Abstract

Small and medium-sized automotive service centers operate in high-turnaround, margin-sensitive environments where administrative inaccuracies directly degrade profitability and customer trust. Common operational bottlenecks include fragmented customer visit histories, unmonitored appointment cancellations, uneven technician bay allocation, unexpected spare parts stockouts during active jobs, client-tamperable invoice calculations, and accidental cascade deletion of financial and service records.

AutoMitra is an open-source, centralized garage management web application engineered specifically to solve these challenges within Indian workshop workflows (localized in Indian Rupees, ■ INR). The system is implemented in Python and Flask using SQLAlchemy ORM, backed natively by MySQL 8.0/MariaDB and embedded SQLite. Key innovations include server-authoritative financial calculation with exact two-decimal Decimal('0.01') quantization, atomic SQL conditional stock deductions preventing concurrent race conditions, restrictive foreign key deletion rules (ON DELETE RESTRICT) protecting historical bills and payment records, full cryptographic CSRF protection, and a 53-item automated regression suite verified across SQLite, live MariaDB/MySQL, and GitHub Actions CI containers.

2. Problem Statement & Core Workshop Bottlenecks

- **1. Duplicate Customer Profiles**: Fragmented records across visits cause conflicting contact details, communication breakdown, and distorted service histories.
- **2. Missed Service Bookings**: Unmonitored appointments lead to vacant bays, unutilized technician hours, and customer churn when follow-ups are omitted.
- **3. Technician Bay Allocation Imbalance**: Lack of real-time visibility into mechanic statuses (Available, Busy, On Leave) and hourly rates causes bay bottlenecks.
- **4. Spare Part Stock Outages & Race Conditions**: Inventory depletion during active repairs stalls workshop turnaround. Python-level non-atomic stock checks permit concurrent requests to drive inventory below zero.

- **5. Billing Inaccuracies & Client Tampering**: Discrepancies between parts installed and items billed, compounded by client-side POST total manipulation and floating-point precision loss.
- **6. Historical Data Loss via Cascade Deletion**: Standard cascading foreign key configurations (ON DELETE CASCADE) silently erase accounting records and payment histories when customer or vehicle records are purged.

3. Existing System vs. Proposed AutoMitra System

Operational Dimension	Legacy Manual / Spreadsheet System	Proposed AutoMitra System
Customer Tracking	Fragmented paper records; phone variations duplicate entries.	Live debounced AJAX duplicate detection on unique phone/email.
Booking Management	Physical registers; missed bookings are forgotten.	Proactive missed booking dashboard with 1-click rescheduling.
Inventory Control	Manual counts; stockouts discovered mid-repair.	Atomic conditional decrement (WHERE stock_quantity >= :qty).
Billing Calculations	Handwritten bills; vulnerable to client-side total manipulation.	Server-authoritative calculations from database labor & parts records.
Currency & Math	Inconsistent rounding; IEEE 754 float drift.	Exact Decimal('0.01') precision localized in Indian Rupees (■).
Historical Preservation	Accidental disposal or cascade deletion of records.	Strict ON DELETE RESTRICT on bills, payments, and bookings.
Security & CSRF	No anti-CSRF protection; raw SQL/stack traces leaked.	Cryptographic CSRF on all 18 forms; sanitized HTTP 500 error pages.
Testing & Verification	Manual spot checking; zero regression safety.	53 automated tests passing on SQLite, live MySQL, and CI.

4. Database Architecture & Relational Schema (9 Tables)

AutoMitra is structured around a centralized relational schema comprising 9 interconnected tables designed to represent workshop operations with strict referential integrity and zero redundancy:

Table Name	Primary / Foreign Keys	Integrity Constraints & Deletion Rules	Key Operational Fields
Customers	PK: id	phone (UNIQUE), email (Indexed)	name, phone, email, address, created_at, updated_at
Vehicles	PK: id FK: customer_id	license_plate (UNIQUE), vin (UNIQUE, NULLable) FK: ON DELETE CASCADE (clean orphan deletion)	license_plate, make, model, year, vin, mileage, fuel_type
Mechanics	PK: id	status IN (Available, Busy, On Leave) hourly_rate >= 0.00	name, phone, specialty, status, hourly_rate (■/hr)
ServiceBookings	PK: id FK: customer_id, vehicle_id	booking_number (UNIQUE) FKs: ON DELETE RESTRICT (preserves history)	booking_number, booking_date, time, service_type, status
ServiceDetails	PK: id FK: booking_id, mechanic_id	booking_id (UNIQUE 1:1 constraint) FK booking: CASCADE; FK mechanic: SET NULL	job_description, diagnosis, labor_hours, labor_cost (■)
SpareParts	PK: id	part_number (UNIQUE SKU) stock_quantity >= 0, prices >= 0.00	part_number, name, category, cost_price, selling_price, stock
PartUsage	PK: id FK: service_detail_id, part_id	quantity > 0, unit_price >= 0.00 FK detail: CASCADE; FK part: RESTRICT	quantity, unit_price (■), subtotal (■), used_at
Bills	PK: id FK: booking_id, customer_id	invoice_number (UNIQUE) FKs: ON DELETE RESTRICT (audit locked)	parts_total, labor_total, tax_amount, discount, grand_total

Payments	PK: id FK: bill_id	transaction_reference (UNIQUE, NULLable) FK bill: ON DELETE RESTRICT	amount (■), payment_method, transaction_reference, notes

5. Database Normalization Proof (1NF to BCNF)

AutoMitra's schema was subjected to rigorous database normalization analysis from First Normal Form (1NF) through Boyce-Codd Normal Form (BCNF) to establish mathematical proof of data redundancy elimination:

- **First Normal Form (1NF)**: All domain values are atomic scalars. Repeating collections (e.g. multiple parts consumed on a job card or multiple partial payments recorded on an invoice) are normalized into independent child entities (PartUsage and Payments).
- **Second Normal Form (2NF)**: Every table utilizes a single-attribute surrogate primary key id ($|PK| = 1$). Since no composite keys exist, proper subsets of candidate keys are empty sets, eliminating the mathematical possibility of partial key dependencies.
- **Third Normal Form (3NF)**: Non-prime attributes depend solely and directly on primary keys without transitive dependencies ($X \to Y \to Z$). Customer profiles are isolated from vehicle entities; job cards capture point-in-time unit prices without replicating catalog metadata; invoices record immutable historical financial totals.
- **Boyce-Codd Normal Form (BCNF)**: For every non-trivial functional dependency $X \to Y$, determinant X is a superkey. Attributes possessing unique functional determination (e.g. Customers.phone, Vehicles.license_plate, SpareParts.part_number, ServiceBookings.booking_number, Bills.invoice_number) are defined with database UNIQUE constraints.

6. Key Engineering Implementations & Vulnerability Remediation

6.1 Concurrency-Safe Atomic Stock Decrement

To prevent race conditions where concurrent requests cause negative stock quantities, AutoMitra replaces Python-level validation with atomic conditional SQL updates: UPDATE SparePart SET stock_quantity = stock_quantity - :qty WHERE id = :id AND stock_quantity >= :qty. If another thread consumes available stock, zero rows are updated, triggering an immediate database rollback and user alert.

6.2 Authoritative Billing & Exact Decimal Quantization

Client browser POST payloads cannot be trusted with monetary values. AutoMitra calculates all invoices authoritatively on the server from raw database labor hours and part usages. Calculations utilize Python's Decimal('0.01') with ROUND_HALF_UP, eliminating binary floating-point rounding errors and strictly bounding taxes (0–100%) and discounts.

6.3 Restrictive Historical Deletion Rules (ON DELETE RESTRICT)

Financial auditability requires that paid invoices and service histories are never deleted. Foreign keys on Payments.bill_id, Bills.booking_id, and ServiceBookings.customer_id enforce ON DELETE RESTRICT. In SQLite development environments, SQLAlchemy engine listeners explicitly enforce this via PRAGMA foreign_keys=ON.

6.4 Empty-String Unique Constraint & Safe Migration Protocol

In MySQL InnoDB unique indexes, multiple NULLs are permitted, but empty strings ('') are treated as duplicate values, resulting in error 1062 during unique constraint creation. AutoMitra's migration pre-flight checks inspect all non-NULL references (WHERE transaction_reference IS NOT NULL GROUP BY transaction_reference HAVING COUNT(*) > 1) and provide auditable procedures to safely convert unprovided references to NULL prior to schema alteration.

6.5 Safe MySQL Test Database Lifecycle

To prevent automated tests from ever destroying or colliding with pre-existing development databases, SafeMySQLTestDatabase generates UUID-isolated database names, queries INFORMATION_SCHEMA.SCHEMATA before DDL execution, tracks test ownership, and guarantees administrative connection closure under all error and collision paths.

7. Comprehensive Automated Test Verification Results

The AutoMitra test suite consists of **53 automated regression and integration tests** implemented in test_app.py. The entire suite executes in ~8.0 seconds locally and 43 seconds on GitHub Actions CI with a MySQL 8.0 service container, achieving a **100% pass rate with zero warnings**:

Subsystem Category	Tests Run	Key Test Cases Verified	Status
Seeding & Core Logic	3	Table creation, duplicate customer detection, missed booking detection	**PASSED**
Inventory & Concurrency	6	Atomic stock decrements, low stock alerts, multithreaded concurrency, restock bounds	**PASSED**
Authorization & Bounds	8	Cross-job IDOR rejection, vehicle-customer ownership, boundary validations	**PASSED**
Financial & Billing	9	Authoritative calculations, Decimal math, bill editing lock, overpayment rejection	**PASSED**
Historical Preservation	11	ON DELETE RESTRICT on bills & payments, booking status locks on paid bills	**PASSED**
Security & Infrastructure	10	CSRF token enforcement across 18 forms, production 500 error sanitization, SQLite mode	**PASSED**
MySQL Integration & Lifecycle	6	SafeMySQLTestDatabase lifecycle, live MySQL DDL, non-destructive migration verification	**PASSED**
Total Verified Suite	**53**	**All functional, security, financial, and integration criteria met**	**100% PASS**

8. Operational Boundaries, Limitations & Future Work

Current Architectural Limitations:
1. **Single-Operator Design**: AutoMitra is currently engineered as an internal service desk console without multi-user role-based authentication (RBAC).
2. **Network Deployment Constraint**: The system must not be exposed directly to the public internet without an authenticating reverse proxy (Cloudflare Access, Nginx HTTP Auth, or Tailscale VPN).
3. **Diagnostic Hardware**: Vehicle diagnostic data is entered manually; direct physical OBD-II scanning interfaces are not currently embedded.

Proposed Future Enhancements:
1. **Role-Based Authentication**: Integration of Flask-Login and bcrypt password hashing to distinguish Mechanic, Cashier, and Administrator views.
2. **Automated Customer Messaging**: WhatsApp Business and SMS gateways for appointment confirmations, status alerts, and invoice delivery.
3. **Digital Payment Gateway Integration**: Razorpay / UPI Intent deep-linking for seamless mobile customer payments.
4. **Predictive Maintenance Analytics**: Mileage-interval algorithms to proactively notify customers of scheduled component replacements.

9. Conclusion

The AutoMitra Vehicle Service Center Management System demonstrates how disciplined relational database engineering, authoritative financial math, and atomic concurrency controls successfully eliminate operational inefficiencies in automotive workshops. By enforcing structural data preservation and comprehensive test-driven quality assurance, AutoMitra establishes a resilient, production-ready foundation for modern garage management.